ORSYS formation

Développer avec Spring Boot.

Tour de table des stagiaires.

- Nom & prénom
- Expériences
- Précisez vos attentes pour cette formation

Présentation du formateur

- **Ingénieur Logiciel en ESN**
  > Capgemini Technology Services > SIGMA
- **Consultant Freelance**
  > Secteur public / TPE-PME
- **Formateur**
  > ORSYS Formation
  > FITEC
  > AFPA
  > PMN
- **Co-founder ELITIS CONSULTING**

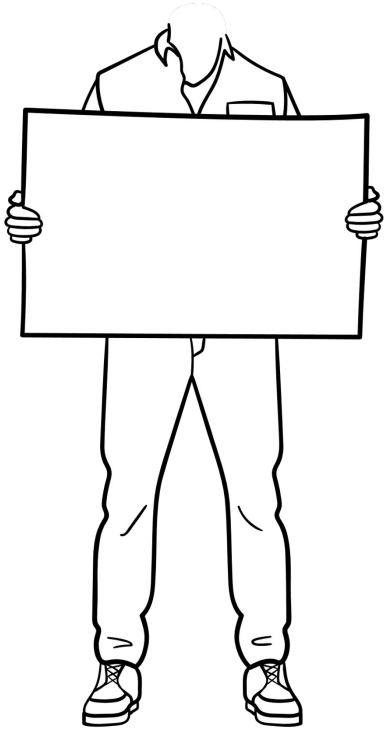
Les objectifs pédagogiques

À l'issue de la formation, le participant sera en mesure de :

- Démarrer un projet Spring Boot
- Maîtriser la configuration automatique de Spring Boot
- Adapter la configuration aux besoins spécifiques des projets
- Simplifier le développement des tests
- Packager et déployer une application Spring Boot selon différents contextes

Table des matières

1. Rappels sur la configuration **Spring** classique.
2. Initialisation d'une application Spring Boot
3. Gestion des dépendances et de l'auto-configuration.
4. Concevoir une application **Spring Boot.**
5. Tester une application **Spring Boot.**
6. Packager et déployer une application **Spring Boot.**
7. Introduction à **Spring Cloud** et aux architectures micro-services.



Rappel : Programmation Orientée Aspect.

Programmation orientée aspect.

Paradigme de programmation qui permet de traiter séparément les préoccupations transversales qui relèvent de la technique des préoccupations ***"métiers"*** qui sont au cœur de l'application.

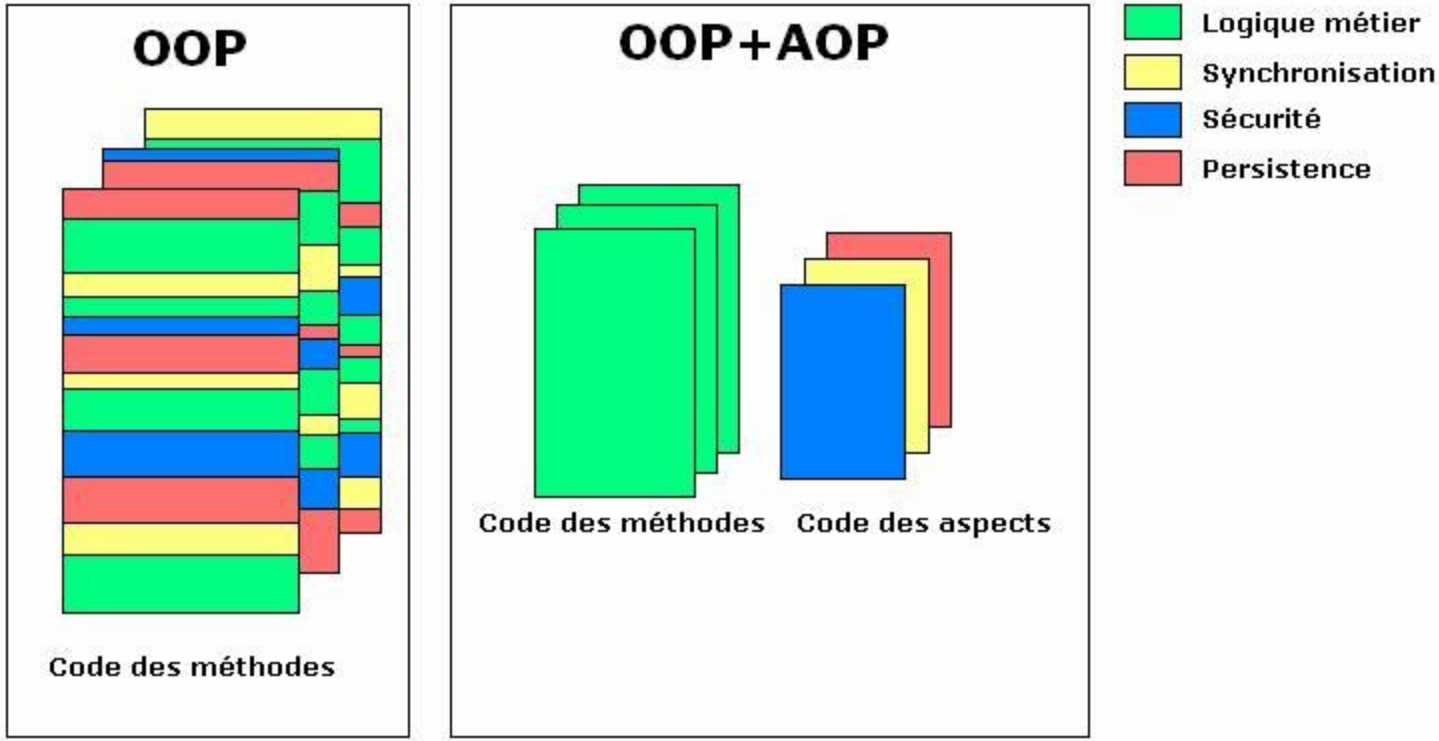
Ce paradigme peut être appliqué sur des langages tels que le **JAVA**, **C++** ou le **C#**.

Programmation orientée aspect.

L'objectif de la **POA** n'est pas de se substituer aux autres paradigmes de programmation tel que l'orienté objet mais de les améliorer pour résoudre les problématiques liées à ces paradigmes.

La **POA** présente les avantages suivants :

- **Gain de productivité**
- **Amélioration de la lisibilité du code**


OOP
Code des méthodes
OOP+AOP
Code des méthodes
Code des aspects
Logique métier
Synchronisation
Sécurité
Persistence

Inversion de contrôle et injection de dépendances.

L'inversion de contrôle est le design pattern commun à tous les frameworks.

Le principe sous-jacent est le suivant :

Le flot d'exécution d'un logiciel n'est plus sous le contrôle direct de l'application elle-même mais du framework.

L'injection de dépendance est l'une des formes les plus connues d'inversion de contrôle.

Exemple sans inversion de contrôle.

```
public void virement(int c1, int c2, double mt) {
    /* Création d'une transaction */
    EntityTransaction transaction=entityManager.getTransaction();
    /* Démarrer la transaction */
    transaction.begin();
    try {
```

Code Technique

```
        /* Code métier */
        retirer(c1,mt);
        verser(c2,mt);
```

Code Métier

```
        /* Valider la transaction  */
        transaction.commit();
    } catch (Exception e) {
        /* Annuler la transaction en cas d'exception  */
        transaction.rollback();
        e.printStackTrace();
    }
}
```

Code Technique

Exemple avec inversion de contrôle.

```
@Transactional
public void virement(int c1, int c2, double mt) {
        retirer(c1,mt);
        verser(c2,mt);
}
```

Code Métier

L'annotation **@Transactional**, permet de déléguer la gestion des transactions au conteneur Spring IOC

Qu’est-ce qu’un framework ?

Le framework prend en charge l’essentiel des aspects techniques, ce qui permet aux programmeurs de se concentrer sur les aspects métier.

Il existe des frameworks de toutes sortes. On en trouve notamment pour la réalisation de tests unitaires ***(JUnit, Jasmin, etc.)***, pour la persistance des données ***(JPI, Hibernate, Entity Framework, etc.)*** et, bien sûr, pour la réalisation d'applications.

Qu’est-ce qu’un framework ?

Un framework d’application est un framework qui dispose d’une classe qui représente une application et qui en fournit une implémentation de base.

La réalisation d’une application consiste alors, pour l'essentiel, à étendre d’une manière ou d’une autre l’application de base fournie par le framework.

Cela peut se faire à l’aide de **l’héritage** ou par tout autre moyen.

Initialisation d’une application Spring Boot.

L’écosystème Spring & sa complexité.

Spring Framework est une plateforme puissante et polyvalente pour le développement d'applications Java. Cependant, en raison de sa richesse et de sa flexibilité, il peut rapidement devenir complexe à configurer.

Éléments de complexité dans Spring :

- **Configuration XML :** Avant **Spring Boot**, la configuration se faisait principalement à l'aide de fichiers XML, ce qui pouvait devenir verbeux et difficile à gérer.
- **Annotations : Spring** a introduit les annotations pour simplifier la configuration, mais leur utilisation intensive peut rendre le code difficile à lire.
- **Gestion des dépendances :** Choisir et configurer les nombreuses bibliothèques nécessaires pour une application peut être fastidieux.

Principe de "convention over configuration".

Le principe de ***"convention over configuration"*** (se traduit *"convention plutôt que configuration"*) signifie que l'on privilégie des conventions par défaut pour réduire la nécessité de configurations explicites.

Spring Boot applique ce principe en fournissant des valeurs par défaut intelligentes et une auto-configuration pour la plupart des scénarios courants.

Quels avantages ? :

- **Simplicité :** Moins de fichiers de configuration et de code boilerplate.
- **Productivité :** Les développeurs peuvent se concentrer sur la logique métier plutôt que sur la configuration.
- **Facilité de démarrage :** Les nouvelles applications peuvent être démarrées rapidement avec peu de configuration initiale.

Starters et auto-configuration.

Starters :

- Les ***"starters"*** de **Spring Boot** sont des dépendances préconfigurées qui regroupent plusieurs bibliothèques nécessaires pour des fonctionnalités spécifiques. Par exemple, **spring-boot-starter-web** inclut les dépendances pour le développement d'applications web avec **Spring MVC**, **Jackson** (pour JSON), et d'autres.

Auto-configuration :

- L'auto-configuration de **Spring Boot** analyse les dépendances présentes dans le classpath et configure automatiquement les beans Spring nécessaires. Par exemple, si **spring-boot-starter-data-jpa** est présent, **Spring Boot** configure automatiquement une DataSource et une **EntityManagerFactory**.

Quelques exemples de starters.

- **spring-boot-starter-web** : pour les applications web.
- **spring-boot-starter-data-jpa** : pour les applications utilisant **JPA** avec **Hibernate**.
- **spring-boot-starter-security** : pour intégrer **Spring Security**.

Mise en oeuvre de Spring Initializer.

Spring Initializer est un outil web qui simplifie la création de projets **Spring Boot**. Il permet de configurer rapidement une application en choisissant les dépendances nécessaires et génère un squelette de projet prêt à l'emploi.

Étapes pour initialiser une application **Spring Boot** avec **Spring Initializr** :

Configurer le projet :

- **Project Metadata :** définir les informations de base telles que Group, Artifact, Name, Description, et Package Name.
- **Project :** choisir entre Maven et Gradle comme outil de build.
- **Language** : choisir le langage (Java, Kotlin, Groovy).
- **Spring Boot Version :** choisir la version de Spring Boot.

Ajouter les dépendances nécessaires pour votre projet (comme **Spring Web**, **Spring Data JPA**, **Thymeleaf**, etc.).

Project

- ○ Gradle - Groovy
- ○ Gradle - Kotlin
- ● Maven

Language

- ● Java
- ○ Kotlin
- ○ Groovy

Spring Boot

- ○ 3.3.1 (SNAPSHOT)
- ○ 3.3.0
- ○ 3.2.7 (SNAPSHOT)
- ● 3.2.6

Project Metadata

Group	com.entreprise
Artifact	kstreams
Name	kstreams
Description	Demo project for Spring Boot
Package name	com.entreprise.kstreams
Packaging	● Jar ○ War
Java	○ 22 ○ 21 ● 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring for Apache Kafka MESSAGING
Publish, subscribe, store, and process streams of records.

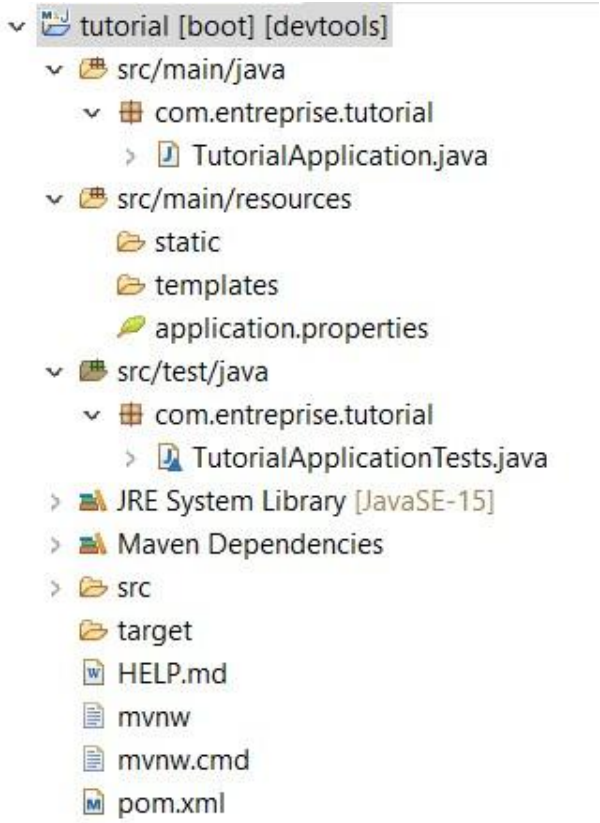
Spring for Apache Kafka Streams MESSAGING
Building stream processing applications with Apache Kafka Streams.

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

GENERATE CTRL + ⏎ | EXPLORE CTRL + SPACE | SHARE...

Présentation de la structure d'un projet généré par Spring Initializer

DEMO



Gestion des dépendances & de l'auto-configuration.

Gestion des Dépendances et de l’Auto-Configuration.

Spring Boot simplifie la gestion des dépendances et la configuration de votre application grâce à des mécanismes **d'auto-configuration** et des conventions prédéfinies.

Structuration du code, les bonnes pratiques.

Séparer les couches de l'application :

Controller : Gérer les requêtes HTTP.

Service : Contient la logique métier.

Repository : Interagit avec la base de données.

Domain/Model : Contient les classes de modèle.

Travaux-pratiques :
Présentation des annotations
Classes d'auto-configuration
Réorganisation des packages.

Concevoir une application Spring Boot.

Spring-Boot KESAKO ?

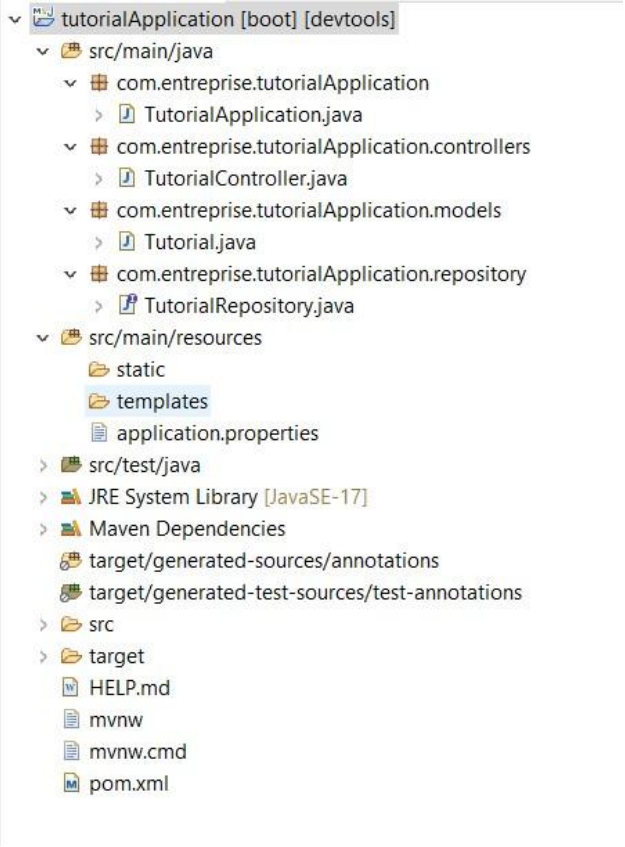
Spring Boot est un **framework** de développement applicatif Java open source.

Il est particulièrement recommandé pour le développement **d'API**.

Ses atouts sont nombreux :

Facilite le développement d'applications complexes
Facilite l'extrême injection de dépendances
Réduire au maximum les fichiers de configuration
Faciliter la gestion des dépendances Maven
Créer des applications autonomes
Fournir un conteneur embarqué (Tomcat)

Structure du projet.



Fichier pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.0.1</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.entreprise</groupId>
    <artifactId>tutorial</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>tutorial</name>
    <description>Spring Boot project tuto</description>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
```

Fichier application.properties

Configuration de l'accès à la base de données postgresql.

```
#DataSource Settings
spring.datasource.url= jdbc:postgresql://localhost:5432/spring-boot-jpa-postgresqlDB
spring.datasource.username= postgres
spring.datasource.password= xxxxxxx

spring.jpa.properties.hibernate.jdbc.lob.non_contextual_creation= true
spring.jpa.properties.hibernate.dialect= org.hibernate.dialect.PostgreSQLDialect

# Hibernate ddl auto (create, create-drop, validate, update)
spring.jpa.hibernate.ddl-auto= create
```

Fichier application.properties

Configuration de l'accès à la base de données MySQL.

```
spring.datasource.url=jdbc:mysql://localhost:3306/testdb?useSSL=false
spring.datasource.username=root
spring.datasource.password=123456

spring.jpa.properties.hibernate.dialect= org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=update
```

Spring-Boot Application

Fichier principal d'application.

```
@SpringBootApplication
public class TutorialApplication {

    public static void main(String[] args) {
        SpringApplication.run(TutorialApplication.class, args);
    }

}
```

Fichier entité représentant la classe Tutorial

```java
package com.entreprise.tutorialApplication.models;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.Size;

@Entity
@Table(name = "tutorial")
public class Tutorial {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long id;

    @Column(name = "title")
    @Size(max = 120)
    private String title;


    @Column(name = "auther")
    @Size(max = 50)
    private String auther;

    @Column(name = "published")
    private boolean published;


    public Tutorial() {

    }
```

```java
    public Tutorial() {

    }

    public Tutorial(String title,String auther) {
        this.title=title;
        this.auther=auther;
        this.published=false;
    }

    public Tutorial(long id, @Size(max = 120) String title, @Size(max = 50) String auther, boolean published) {
        super();
        this.id = id;
        this.title = title;
        this.auther = auther;
        this.published = published;
    }
```

Fichier représentant la l'interface Tutorial Repository

```java
package com.entreprise.tutorialApplication.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.entreprise.tutorialApplication.models.Tutorial;

@Repository
public interface TutorialRepository extends JpaRepository<Tutorial, Long> {
    List<Tutorial> findbyPublished(boolean published);
    List<Tutorial> findbyTitle(String title);
}
```

Fichier représentant la classe Tutorial Controller

```java
package com.entreprise.tutorialApplication.controllers;

import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.entreprise.tutorialApplication.models.Tutorial;
import com.entreprise.tutorialApplication.repository.TutorialRepository;

@RestController
@RequestMapping(path = "/api/")
public class TutorialController {

    @Autowired
    TutorialRepository tutorialRepository;

    @GetMapping(path = "/tutorials")
    public List<Tutorial> getTutorials(){
        List<Tutorial> liste_de_tutoriels = (List<Tutorial>) tutorialRepository.findAll();
        if (!liste_de_tutoriels.isEmpty()) {
            return liste_de_tutoriels;
        }

        return null;
    }

    @GetMapping(path = "/tutorials/{id}")
    public Tutorial getTutorialByID(@PathVariable Long id) {
        Optional<Tutorial> retrievedTutorial = tutorialRepository.findById(id);
        if(retrievedTutorial.isPresent()) {
            return retrievedTutorial.get();
        }
        return null;
    }


    @PostMapping(path = "/tutorials")
    public Tutorial addTutorial(@RequestBody Tutorial tutorial) {
        Tutorial _tutorial = tutorialRepository.save(new Tutorial(tutorial.getTitle(), tutorial.getAuthor()));
        return _tutorial;
    }

    @DeleteMapping(path = "/tutorials/{id}")
    public HttpStatus deleteTutorialByID(@PathVariable Long id) {
        tutorialRepository.deleteById(id);
        return HttpStatus.OK;
    }

    @DeleteMapping(path = "/tutorials")
    public HttpStatus deleteTutorials() {
        tutorialRepository.deleteAll();;
        return HttpStatus.OK;
    }

}
```

Présentation de Postman

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and **unlock all free features in Postman.**

History New Import

Today

Yesterday

GET localhost:8080/api/

localhost:8080/api/ Save

GET localhost:8080/api/ Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Response

Could not send request

Error: connect ECONNREFUSED 127.0.0.1:8080 | View in Console

Learn more about troubleshooting API requests

Console Not connected to a Postman account

Conventions et bonnes pratiques.

1. Conception RESTful (pour les API REST) :

Utiliser les Méthodes HTTP Appropriées : **GET** pour récupérer des données, **POST** pour créer, **PUT/PATCH** pour mettre à jour, et **DELETE** pour supprimer.

Ressources Nommées de Manière Claire : Utiliser des noms de ressources (au pluriel) clairs et descriptifs dans les **URLs**.

Gestion des Réponses : Retourner des codes de réponse **HTTP** appropriés (par exemple, **200 OK**, **404 Not Found**).

Conventions et bonnes pratiques.

2. Standardisation :

Formats de Réponse Consistents : Utiliser un format standard *(comme JSON)* pour toutes les réponses.

3. Sécurité

Authentification et Autorisation : Implémenter des mécanismes robustes comme OAuth, JWT.
Validation et Sanitisation des Données : Toujours valider les données entrantes pour prévenir les attaques telles que l'injection SQL.

Conventions et bonnes pratiques.

4. Documentation:

Documenter l'API : Fournir une documentation complète et à jour, idéalement avec des exemples d'utilisation.
Utiliser des Outils de Documentation : Comme Swagger ou Redoc pour générer une documentation interactive.

5. Gestion des Erreurs

Messages d'Erreur Clairs : Fournir des messages d'erreur descriptifs avec des codes d'erreur standardisés.
Documentation des Erreurs : Documenter les types d'erreurs possibles et leur signification.

Conventions et bonnes pratiques.

6. Performances et Scalabilité

Pagination : Pour les grandes collections de données, utiliser la pagination pour limiter la charge du serveur et améliorer la réactivité.
Mise en Cache : Utiliser des mécanismes de mise en cache pour améliorer les performances.

7. Versioning

Versionner l'API : Prévoir des versions pour l'API pour gérer les changements sans perturber les utilisateurs existants.
Stratégie de Versioning : Utiliser des URL ou des en-têtes pour gérer différentes versions.

Conventions et bonnes pratiques.

8. Respect des Limites de Taux

Limitation des Taux : Mettre en place des limites pour prévenir l'abus et la surcharge des serveurs.

9. Test et Monitoring

Tests Automatisés : Écrire des tests pour chaque aspect de l'API.

Surveillance de l'API : Utiliser des outils de monitoring pour surveiller les performances et la santé de l'API.

10. Prise en Charge des Clients : Etre réceptif aux retours utilisateurs – produire des guides.

API REST avec Swagger

– Pour **Spring Boot 3** :

Pour utiliser Swagger 3 dans votre projet Maven, vous devez ajouter la dépendance ***springdoc-openapi-starter-webmvc-ui*** au fichier *pom.xml* de votre projet :

```
<!-- Ajout swagger-->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.0.3</version>
</dependency>
```

http://localhost:8080/swagger-ui/index.html

Refactoring de l'API.

```java
package com.entreprise.tutorialApplication.controllers;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;


import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.entreprise.tutorialApplication.models.Tutorial;
import com.entreprise.tutorialApplication.repository.TutorialRepository;
```

```java
@RestController
@RequestMapping(path = "/api")
public class TutorialController {

    @Autowired
    TutorialRepository tutorialRepository;


    @GetMapping(path = "/tutorials", produces = "application/json")
    public ResponseEntity<List<Tutorial>> getTutorials(@RequestParam(required = false) String title){
        try {

            List<Tutorial> list_of_tutorials = new ArrayList<Tutorial>();

            if (title == null) {
                tutorialRepository.findAll().forEach(list_of_tutorials::add);
            }

            else {
                tutorialRepository.findByTitle(title).forEach(list_of_tutorials::add);
            }

            if (list_of_tutorials.isEmpty()) {
                return new ResponseEntity<List<Tutorial>>(HttpStatus.NO_CONTENT);
            }

            return new ResponseEntity<>( list_of_tutorials, HttpStatus.OK);

        } catch (Exception e) {
            // TODO: handle exception
            return new ResponseEntity<List<Tutorial>>(HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
```

Refactoring de l'API.

```
@GetMapping(path = "/tutorials/{id}", produces = "application/json")
public ResponseEntity<Tutorial> getTutorialByID(@PathVariable Long id){
    try {
        Optional<Tutorial> retrievedTutorial = tutorialRepository.findById(id);
        if(retrievedTutorial.isPresent()) {
            return new ResponseEntity<Tutorial>(retrievedTutorial.get(),HttpStatus.OK);
        } else {
            return new ResponseEntity<>(HttpStatus.NOT_FOUND);
        }
    } catch (Exception e) {
        // TODO: handle exception
        return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}


@PostMapping(path = "/tutorials", produces = "application/json")
public ResponseEntity<Tutorial> addTutorial(@RequestBody Tutorial tutorial){
    try {
        Tutorial _tutorial = tutorialRepository.save(new Tutorial(tutorial.getTitle(), tutorial.getAuthor()));
        return new ResponseEntity<Tutorial>(_tutorial,HttpStatus.CREATED);
    } catch (Exception e) {
        // TODO: handle exception
        return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Refactoring de l'API.

```
    @DeleteMapping(path = "/tutorials/{id}", produces = "application/json")
    public ResponseEntity<HttpStatus> deleteByID(@PathVariable Long id){
        try {
            tutorialRepository.deleteById(id);
            return new ResponseEntity<HttpStatus>(HttpStatus.OK);
        } catch (Exception e) {
            // TODO: handle exception
            return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }


    @DeleteMapping(path = "/tutorials", produces = "application/json")
    public ResponseEntity<HttpStatus> deleteAll(){
        try {
            tutorialRepository.deleteAll();
            return new ResponseEntity<HttpStatus>(HttpStatus.OK);
        } catch (Exception e) {
            // TODO: handle exception
            return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }

}
```

On souhaite créer une application qui permet de gérer le catalogue des livres appartenant à des catégories différentes.

Chaque Livre est définit par :

- Son titre de type String
- Son auteur de type String
- Son prix de type float
- Sa disponibilité de type boolean

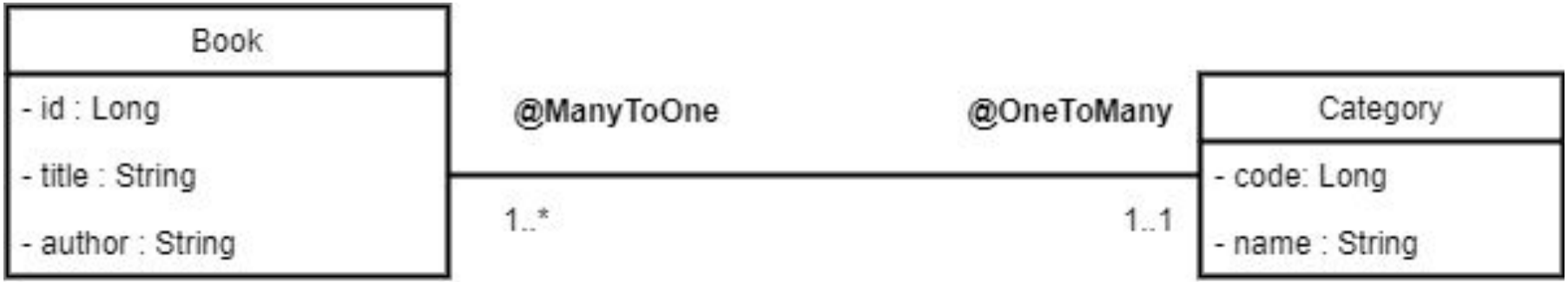
Une catégorie est définie par :

- Son code de type Long (Auto Increment)
- Son nom de type String

L'application doit permettre :

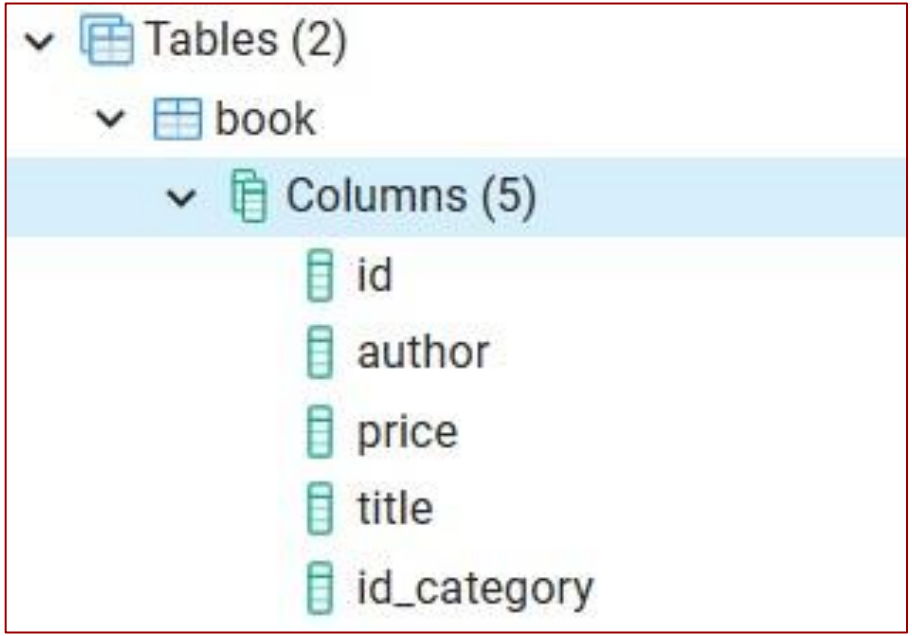
- ❑ D'ajouter une nouvelle catégorie
- ❑ Ajouter un livre appartenant à une catégorie
- ❑ Consulter toutes les catégories
- ❑ Consulter les livres dont le nom contient un mot clé
- ❑ Consulter les livres d'une catégorie
- ❑ Consulter un livre
- ❑ Mettre à jour un livre
- ❑ Supprimer une catégorie

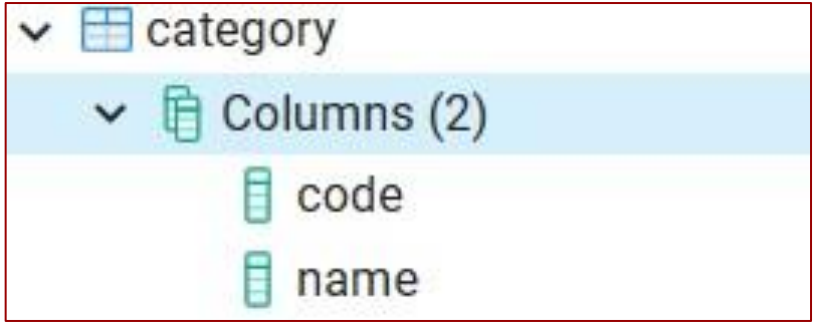
Diagramme de classe : Modèle objet



- CATEGORY (ID_CAT, NOM_CATEGORY)
- BOOK (ID,TITLE, AUTHOR, PRIX, DISPO, #ID_CAT)

Base de données





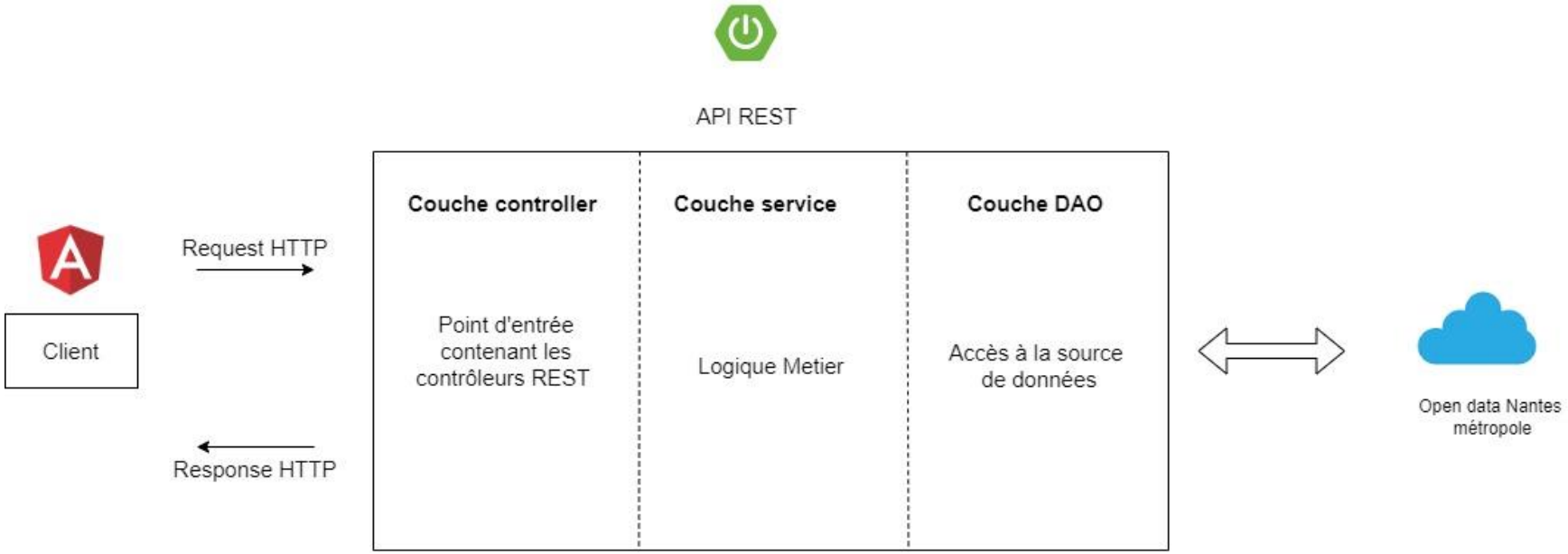
Travaux-Pratiques : Mise en oeuvre d’une API REST

NeoService est l'entreprise (fictive) chargée de l'entretien des toilettes publiques de la ville de Nantes. Elle souhaite développer une application permettant de lister les toilettes de la métropole et d'en consulter la disponibilité.

Vous devez créer une api REST qui renvoie en temps réel la liste des toilettes publiques en vous servant de l’open data de la ville de Nantes disponible à l’adresse :
https://data.nantesmetropole.fr/pages/home/

Bonus : Sécurisez l’accès aux données avec une clé API (API_KEY)

Architecture logicielle



Tester une application Spring Boot.

Les tests logiciels

Test unitaire

permet de vérifier
le bon fonctionnement d'une partie
précise d'un logiciel ou d'une
portion d'un programme.

Test système

Permet d'évaluer la conformité de
l'ensemble du système par rapport
aux exigences spécifiées.

Test d'acceptance

permet d'assurer formellement que le
produit est conforme aux spécifications

Test de régression

permet de détecter les
régressions introduites dans un logiciel
après un changement effectué dans celui-ci.

Test d'intégration

permet de vérifier le bon fonctionnement
d'une partie précise d'un logiciel ou d'une
portion d'un
programme (appelée « unité » ou « module
»)

Beta test

un échantillon du public visé essaie le
produit

Tester une application Spring Boot.

Le test est une partie cruciale du développement d'applications pour garantir la qualité et la fiabilité du logiciel.

Spring Boot offre une multitude de fonctionnalités pour faciliter le test des applications, allant de tests unitaires simples à des tests d'intégration complets.

Auto-Configuration des Tests.

Spring Boot fournit des annotations spécifiques pour simplifier la configuration des tests. Ces annotations permettent de configurer automatiquement le contexte Spring nécessaire pour les différents types de tests.

- **@SpringBootTest** : Charge l'intégralité du contexte de l'application pour les tests d'intégration.
- **@WebMvcTest** : Configure uniquement le contexte Web MVC pour tester les contrôleurs.
- **@DataJpaTest** : Configure un contexte limité pour tester la persistance des données (JPA).

Test de la Persistance avec @DataJpaTest.

L'annotation **@DataJpaTest** est utilisée pour tester les composants **JPA**.

Elle configure automatiquement une base de données en mémoire, un **EntityManager**, et un contexte Spring limité.

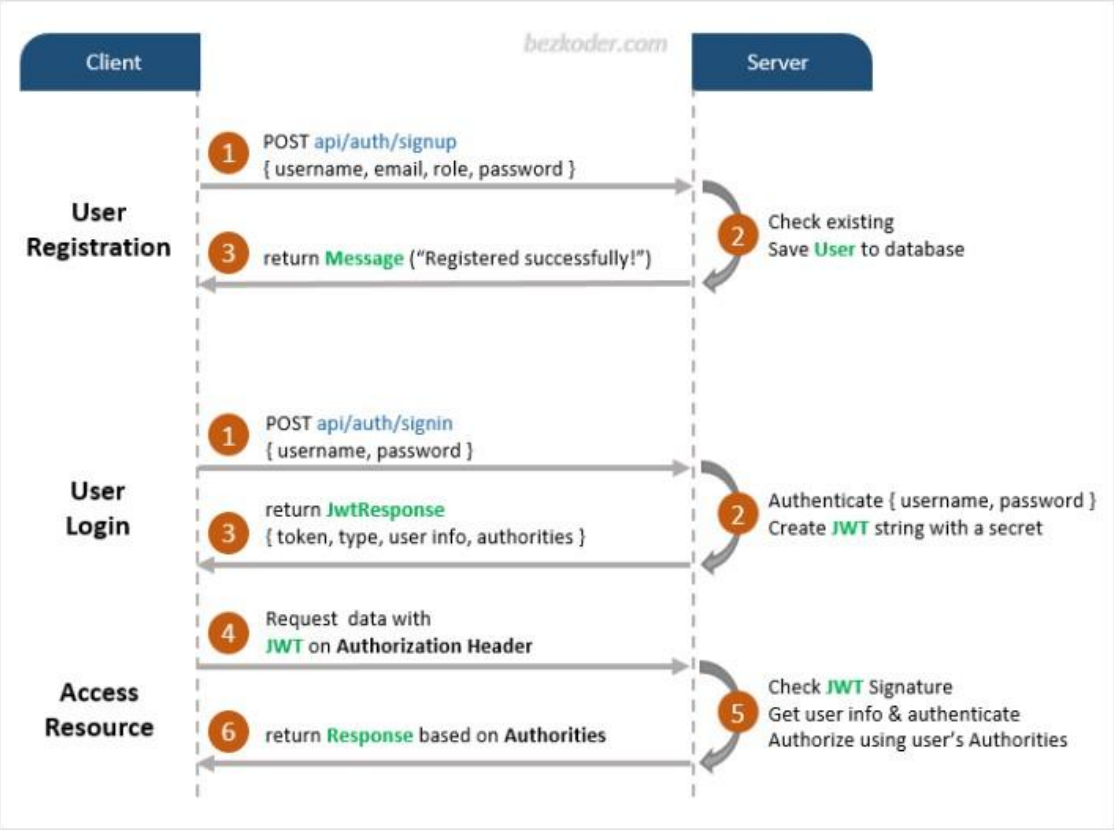
Test de la Persistance avec @DataJpaTest.

```
@RunWith(SpringRunner.class)
@DataJpaTest
public class UserRepositoryTest {
    @Autowired
    private TestEntityManager entityManager;
    @Autowired
    private UserRepository userRepository;
    @Test
    public void testFindByUsername() {
        // Given
        User user = new User("john_doe", "John Doe");
        entityManager.persistAndFlush(user);
        // When
        User found = userRepository.findByUsername(user.getUsername());
        // Then
        assertThat(found.getUsername()).isEqualTo(user.getUsername());
    }
}
```

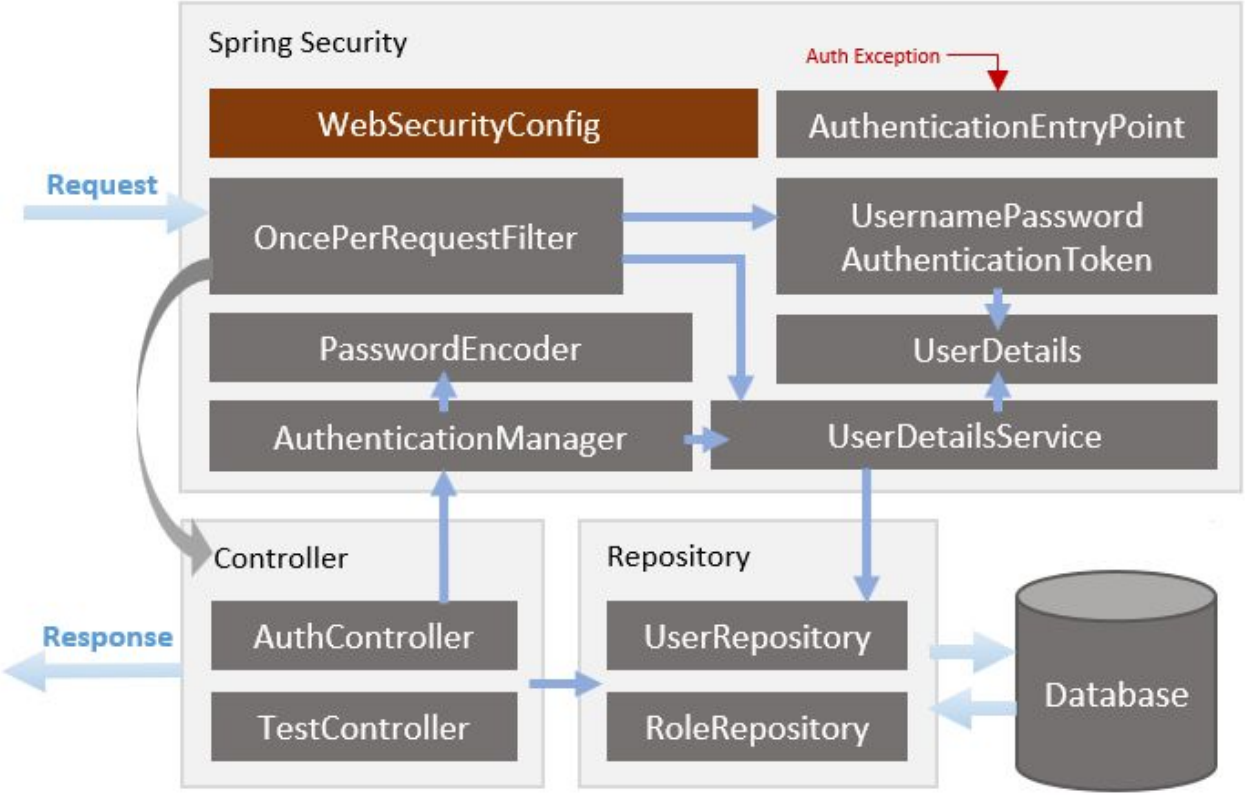
Travaux-pratiques : Réaliser les tests unitaires de l'application développée précédemment.

Rappels sur la sécurité

Sign up and login avec JWT authentification



Authentification basée sur Spring-Boot, Spring security & JWT.



Dépendances nécessaires.

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>

<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>

<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Package models.

Classe énumérée **ERole** pour l'énumération des rôles ou privilèges dans l'application *(ROLE_USER et ROLE_ADMINISTRATOR)*.

```java
package com.entreprise.tutorialApplication.models;

public enum ERole {
    ROLE_USER,
    ROLE_ADMINISTRATOR
}
```

Classe **Role**, pour la gestion des rôles. Les noms reconnus aux rôles sont de type **ERoles**.

```java
package com.entreprise.tutorialApplication.models;

import jakarta.persistence.Column;

@Entity
@Table(name = "roles")
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(length = 20)
    private ERole name;

    public Role() {}

    public Role(ERole name) {
        this.name = name;
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public ERole getName() {
        return name;
    }

    public void setName(ERole name) {
        this.name = name;
    }
}
```

Package models – Classe User.

```java
package com.entreprise.tutorialApplication.models;

import java.util.HashSet;

@Entity
@Table(name = "Users", uniqueConstraints = {
        @UniqueConstraint(columnNames = "username"),
        @UniqueConstraint(columnNames = "email")
    })
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @Column(name = "username")
    @Size(max = 50)
    private String username;

    @Column(name = "email")
    @Size(max = 120)
    @Email
    private String email;

    @Column(name = "password")
    @Size(max = 20)
    private String password;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(name = "user_roles",
                joinColumns = @JoinColumn(name = "user_id"),
                inverseJoinColumns = @JoinColumn(name = "role_id") )
    private Set<Role> roles = new HashSet<Role>();
```

Classe **User** de gestion des utilisateurs. Chaque utilisateur disposant des attributs **username** - **email** - **password** et **liste des rôles**.

Package models – Classe User (suite).

```java
//Getters & Setters

public User() {}

public User(String username, String email, String password) {
    this.username = username;
    this.email = email;
    this.password = password;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getUsername() {
    return username;
}

public void setUsername(String username) {
    this.username = username;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getPassword() {
    return password;
}
```

Package Repository : UserRepository et RoleRepository.

```java
package com.entreprise.tutorialApplication.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.entreprise.tutorialApplication.models.Tutorial;

@Repository
public interface TutorialRepository extends JpaRepository<Tutorial, Long> {
    List<Tutorial> findbyPublished(boolean published);
    List<Tutorial> findbyTitle(String title);
}
```

RoleRepository Responsable de l'accès aux données sur les rôles en BD.

UserRepository Responsable de l'accès aux données utilisateur en BD.

```java
import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;


import com.entreprise.tutorialApplication.models.User;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByusername(String username);
    Boolean existsByUsername(String username);
    Boolean existsByEmail(String email);

}
```

Package security.service – UserDetailsImpl.

```java
package com.entreprise.tutorialApplication.security.service;

import java.util.Collection;
import java.util.List;
import java.util.Objects;
import java.util.stream.Collectors;

import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import com.entreprise.tutorialApplication.models.User;
import com.fasterxml.jackson.annotation.JsonIgnore;

public class UserDetailsImpl implements UserDetails {

    private static final long serialVersionUID = 1L;

    private Long id;

    private String username;

    private String email;

    @JsonIgnore
    private String password;

    private Collection<? extends GrantedAuthority> authorities;


    public UserDetailsImpl(Long id, String username, String email,
            String password, Collection<? extends GrantedAuthority> authorities ) {
        this.id = id;
        this.username = username;
        this.email = email;
        this.password = password;
        this.authorities = authorities;
    }
```

UserDetails contient les informations nécessaires ***(telles que : nom d'utilisateur, mot de passe, autorités)*** pour créer un objet d'authentification.

Package security.service – UserDetailsImpl (Suite).

```
public static UserDetailsImpl build(User user) {
    List<GrantedAuthority> authorities = user.getRoles().stream().map(role -> new SimpleGrantedAuthority(role.getName().name()))
            .collect(Collectors.toList());

    return new UserDetailsImpl(
            user.getId(),
            user.getUsername(),
            user.getEmail(),
            user.getPassword(),
            authorities);
}

@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    // TODO Auto-generated method stub
    return null;
}

public Long getID() {
    return id;
}

public void setID(Long id) {
    this.id = id;
}


public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}
```

Package security.service – UserDetailsImpl (Suite).

```
public boolean equals(Object o) {
    if (this == o) {
        return true;
    }
    if (o == null || getClass()!= o.getClass() ) {
        return false;

    }
    UserDetailsImpl user = (UserDetailsImpl) o;
    return Objects.equals(id, user.id);
}
```

Package security.service – UserDetailsServiceImpl.

```java
package com.entreprise.tutorialApplication.security.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import com.entreprise.tutorialApplication.models.User;
import com.entreprise.tutorialApplication.repository.UserRepository;

@Service
public class UserDetailsServiceImpl implements UserDetailsService  {

    @Autowired
    UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        // TODO Auto-generated method stub
        User user = userRepository.findByusername(username)
                .orElseThrow( () -> new UsernameNotFoundException("User not found with username" + username));


        return UserDetailsImpl.build(user);

    }

}
```

L'interface **UserDetailsService** dispose d'une méthode pour charger l'utilisateur par nom d'utilisateur et renvoie un objet **UserDetails** que **Spring Security** peut utiliser pour l'authentification et la validation.

Package security.jwt – AuthenticationEntryPoint

```java
import java.io.IOException;

@Component
public class AuthEntryPointJwt implements AuthenticationEntryPoint {

  private static final Logger logger = LoggerFactory.getLogger(AuthEntryPointJwt.class);

  @Override
  public void commence(HttpServletRequest request, HttpServletResponse response, AuthenticationException authException)
      throws IOException, ServletException {
    logger.error("Unauthorized error: {}", authException.getMessage());

    response.setContentType(MediaType.APPLICATION_JSON_VALUE);
    response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);

    final Map<String, Object> body = new HashMap<>();
    body.put("status", HttpServletResponse.SC_UNAUTHORIZED);
    body.put("error", "Unauthorized");
    body.put("message", authException.getMessage());
    body.put("path", request.getServletPath());

    final ObjectMapper mapper = new ObjectMapper();
    mapper.writeValue(response.getOutputStream(), body);
  }

}
```

AuthenticationEntryPoint détectera l'erreur d'authentification.

Package security.jwt – AuthTokenFilter.

```java
package com.entreprise.tutorialApplication.security.jwt;

import org.slf4j.Logger;

public class AuthTokenFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtils jwtUtils;


    private static final Logger logger = LoggerFactory.getLogger(AuthTokenFilter.class);

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
            throws ServletException, IOException {
        // TODO Auto-generated method stub
        try {
            String jwt = parseJwt(request);


        } catch (Exception e) {
            // TODO: handle exception
        }

    }

    private String parseJwt(HttpServletRequest request) {
        // TODO Auto-generated method stub
        String headerAuth = request.getHeader("Authorization");
        if(StringUtils.hasText(headerAuth) && headerAuth.startsWith("Beaarer")) {
            return headerAuth.substring(7);
        }

        return null;
    }

}
```

OncePerRequestFilter effectue une seule exécution pour chaque requête adressée à notre **API**.

Il fournit une méthode **doFilterInternal()** que nous implémenterons en analysant et en validant **JWT**, en chargeant les détails de l'utilisateur *(à l'aide de UserDetailsService)*, en vérifiant l'autorisation *(à l'aide de UsernamePasswordAuthenticationToken)*.

Package security.jwt – JwtUtils.

```java
package com.entreprise.tutorialApplication.security.jwt;

import java.util.Date;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Component;

import com.entreprise.tutorialApplication.security.service.UserDetailsImpl;

import io.jsonwebtoken.ExpiredJwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.MalformedJwtException;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.UnsupportedJwtException;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;

import java.security.Key;
```

```java
@Component
public class JwtUtils {

private static final Logger logger = LoggerFactory.getLogger(JwtUtils.class);

    @Value("${orsys.app.jwtSecret}")
    private String jwtSecret;

    @Value("${orsys.app.jwtExpirationMs}")
    private int jwtExpirationMs;

    public String generateJwtToken(Authentication authentication ) {

        UserDetailsImpl userPrincipal = (UserDetailsImpl) authentication;

        return Jwts.builder()
                .setSubject((userPrincipal.getUsername()))
                .setIssuedAt(new Date())
                .setExpiration(new Date((new Date()).getTime() + jwtExpirationMs ))
                .signWith(Key(), SignatureAlgorithm.HS256)
                .compact();

    }

    private Key Key() {
        // TODO Auto-generated method stub
        return Keys.hmacShaKeyFor(Decoders.BASE64.decode(jwtSecret));
    }
```

Package security.jwt – JwtUtils.

```
public String getUserNameFromJwtToken(String token) {
    return Jwts.parserBuilder().setSigningKey(Key()).build()
                .parseClaimsJws(token).getBody().getSubject();
  }

  public boolean validateJwtToken(String authToken) {
    try {
      Jwts.parserBuilder().setSigningKey(Key()).build().parse(authToken);
      return true;
    } catch (MalformedJwtException e) {
      logger.error("Invalid JWT token: {}", e.getMessage());
    } catch (ExpiredJwtException e) {
      logger.error("JWT token is expired: {}", e.getMessage());
    } catch (UnsupportedJwtException e) {
      logger.error("JWT token is unsupported: {}", e.getMessage());
    } catch (IllegalArgumentException e) {
      logger.error("JWT claims string is empty: {}", e.getMessage());
    }

    return false;
    }

}
```

Package configuration – WebSecurityConfig.

```java
package com.entreprise.tutorialApplication.configuration;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

import com.entreprise.tutorialApplication.security.jwt.AuthEntryPointJwt;
import com.entreprise.tutorialApplication.security.jwt.AuthTokenFilter;
import com.entreprise.tutorialApplication.security.service.UserDetailsServiceImpl;
```

Package configuration – WebSecurityConfig.

```java
@Configuration
@EnableMethodSecurity
public class WebSecurityConfig { // extends WebSecurityConfigurerAdapter {
  @Autowired
  UserDetailsServiceImpl userDetailsService;

  @Autowired
  private AuthEntryPointJwt unauthorizedHandler;

  @Bean
  public AuthTokenFilter authenticationJwtTokenFilter() {
    return new AuthTokenFilter();
  }

  @Bean
  public DaoAuthenticationProvider authenticationProvider() {
      DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();

      authProvider.setUserDetailsService(userDetailsService);
      authProvider.setPasswordEncoder(passwordEncoder());

      return authProvider;
  }

  @Bean
  public AuthenticationManager authenticationManager(AuthenticationConfiguration authConfig) throws Exception {
    return authConfig.getAuthenticationManager();
  }

  @Bean
  public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
  }
```

Package configuration – WebSecurityConfig.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  http.csrf(csrf -> csrf.disable())
      .exceptionHandling(exception -> exception.authenticationEntryPoint(unauthorizedHandler))
      .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
      .authorizeHttpRequests(auth ->
        auth.requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/api/test/**").permitAll()
            .anyRequest().authenticated()
      );

  http.authenticationProvider(authenticationProvider());

  http.addFilterBefore(authenticationJwtTokenFilter(), UsernamePasswordAuthenticationFilter.class);

  return http.build();
}
}
```

Travaux-pratiques : Mise en oeuvre d'une infrastructure technique .

Packager et déployer une application Spring Boot.

Création d’un war pour le déploiement classique.

Spring Boot permet de créer des applications qui peuvent être déployées sous forme de WAR (Web Application Archive) pour être déployées sur des serveurs d'applications traditionnels comme Tomcat, JBoss, ou WebLogic.

Étapes pour créer un fichier WAR :

Modifier le fichier pom.xml pour indiquer que le packaging est de type WAR :

<packaging>war</packaging>

Création d'un war pour le déploiement classique.

Ajouter une classe de configuration principale qui étend **SpringBootServletInitializer**:

```
@SpringBootApplication
public class DemoApplication extends SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder application)
        return application.sources(DemoApplication.class);
    }

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Création d’un war pour le déploiement classique.

Exclure l'embedded **Tomcat** si vous utilisez un autre serveur d'application :

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>
</dependency>
```

Construire le fichier WAR : **mvn clean package**

Création d'un JAR "Self-Contained".

Assurez-vous que le packaging est de type JAR dans votre pom.xml (par défaut) :

```
<packaging>jar</packaging>
```

Ajouter le plugin Spring Boot Maven dans le fichier pom.xml :

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

Création d’un JAR “Self-Contained”.

Utilisez Maven pour construire le fichier JAR :

```
mvn clean package
```

Le fichier JAR généré sera situé dans le répertoire target et peut être exécuté directement :

```
java -jar target/application-0.0.1-SNAPSHOT.jar
```

Déploiement dans une Image Docker.

Docker permet de containeriser les applications, ce qui facilite leur déploiement sur différentes plateformes sans souci de compatibilité.

- Une image docker est créée à partir d’un Dockerfile.
- Une image docker comprend plusieurs couches.
- Les différentes couches comprennent l’application et ses dépendances (binaires – bibliothèques…).
- Un conteneur est une image en cours d'exécution.

Créer un Dockerfile

```
# Use a base image with Java
FROM openjdk:17-jdk-alpine
# Add a volume pointing to /tmp
VOLUME /tmp
# The application's JAR file
ARG JAR_FILE=target/application-0.0.1-SNAPSHOT.jar
# Add the application's JAR file to the container
ADD ${JAR_FILE} app.jar
# Run the JAR file
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Créer un Dockerfile.

Construire l'image Docker :

Utilisez la commande Docker pour construire l'image :

docker build -t application-demo .

Exécuter le conteneur Docker :

Utilisez Docker pour exécuter un conteneur basé sur l'image créée :

docker run -p 8080:8080 application-demo

Cette commande expose le port 8080 du conteneur sur le port 8080 de l'hôte.

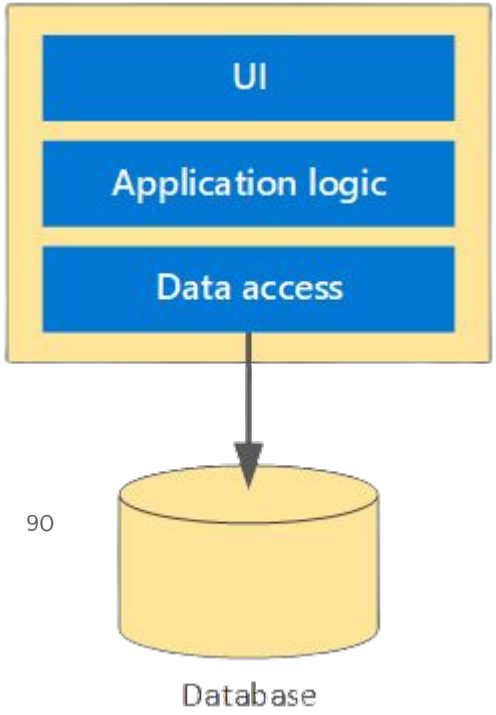
Travaux-pratiques : Produire un jar auto-exécutable et le déployer sur une image docker.

Introduction à Spring Cloud et aux architectures microservices.

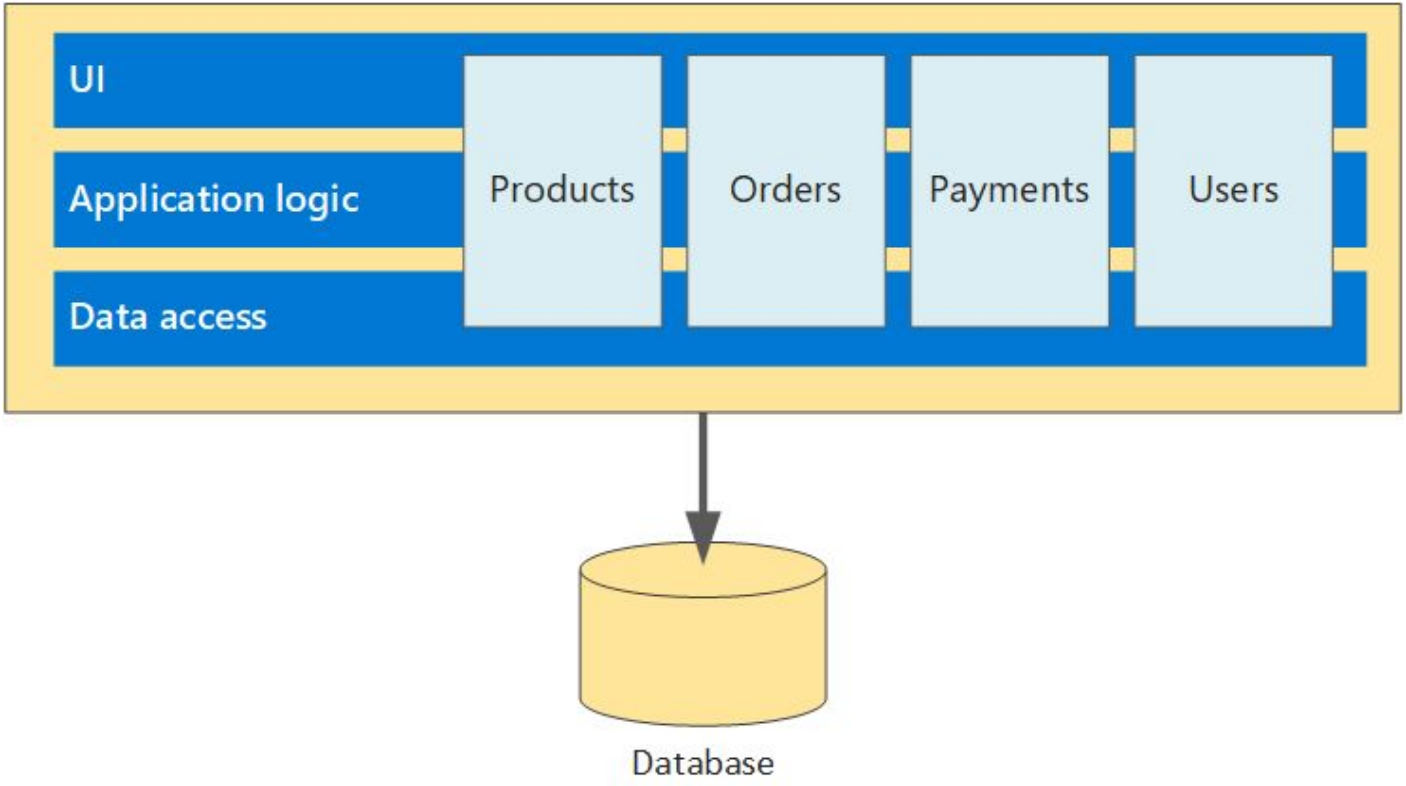
Architecture monolithique.

Une architecture monolithique désigne en informatique,un modèle traditionnel d'applications conçu comme une unité autonome et indépendante d'autres applications.

L'ensemble du code et les fonctionnalités implémentées sont dans un seul programme.



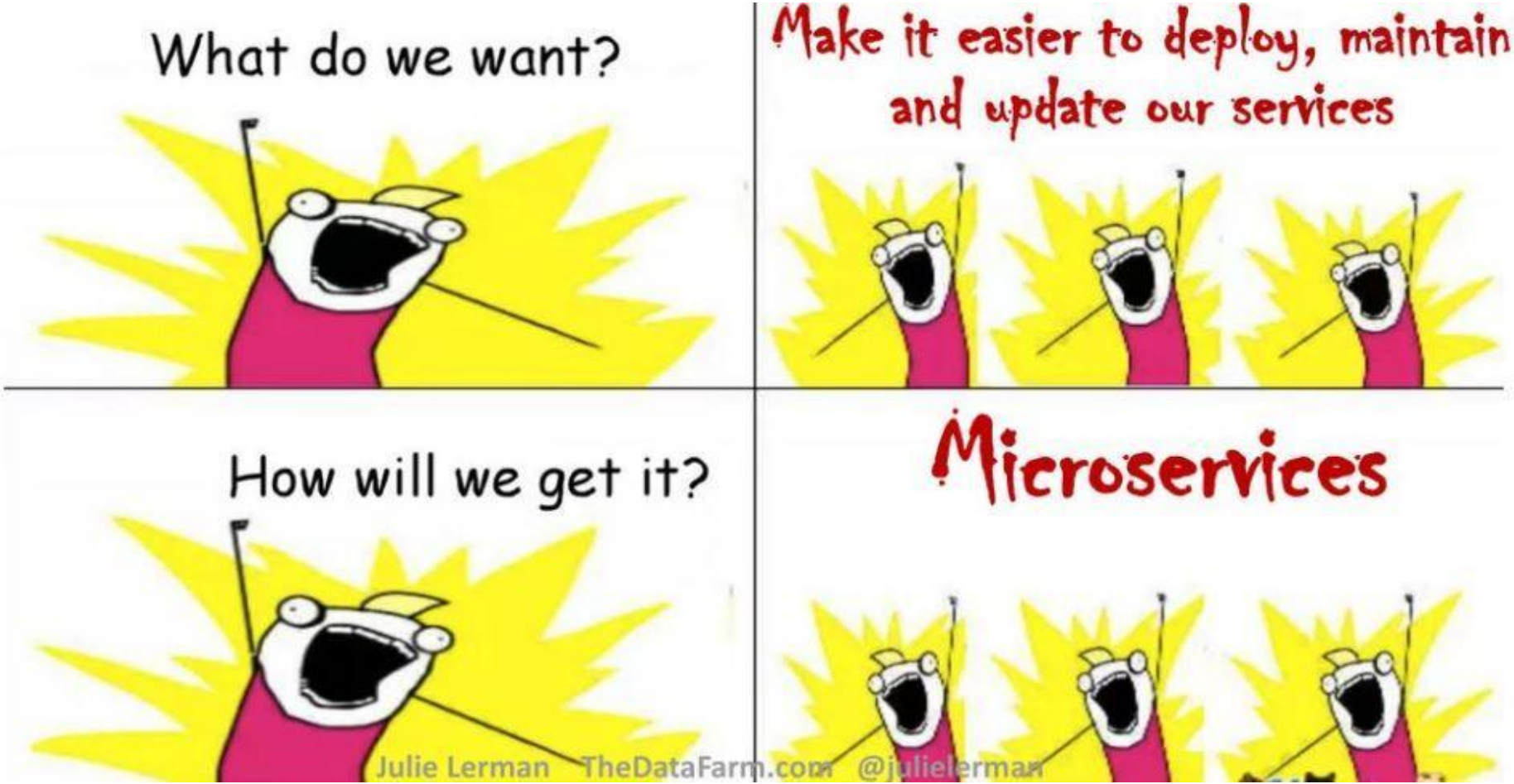
Exemple


UI
Application logic
Data access
Products
Orders
Payments
Users
Database

Avantages – Inconvénients

- Déploiement facile
- Développement
- Performances
- Tests simplifiés
- Débogage facile

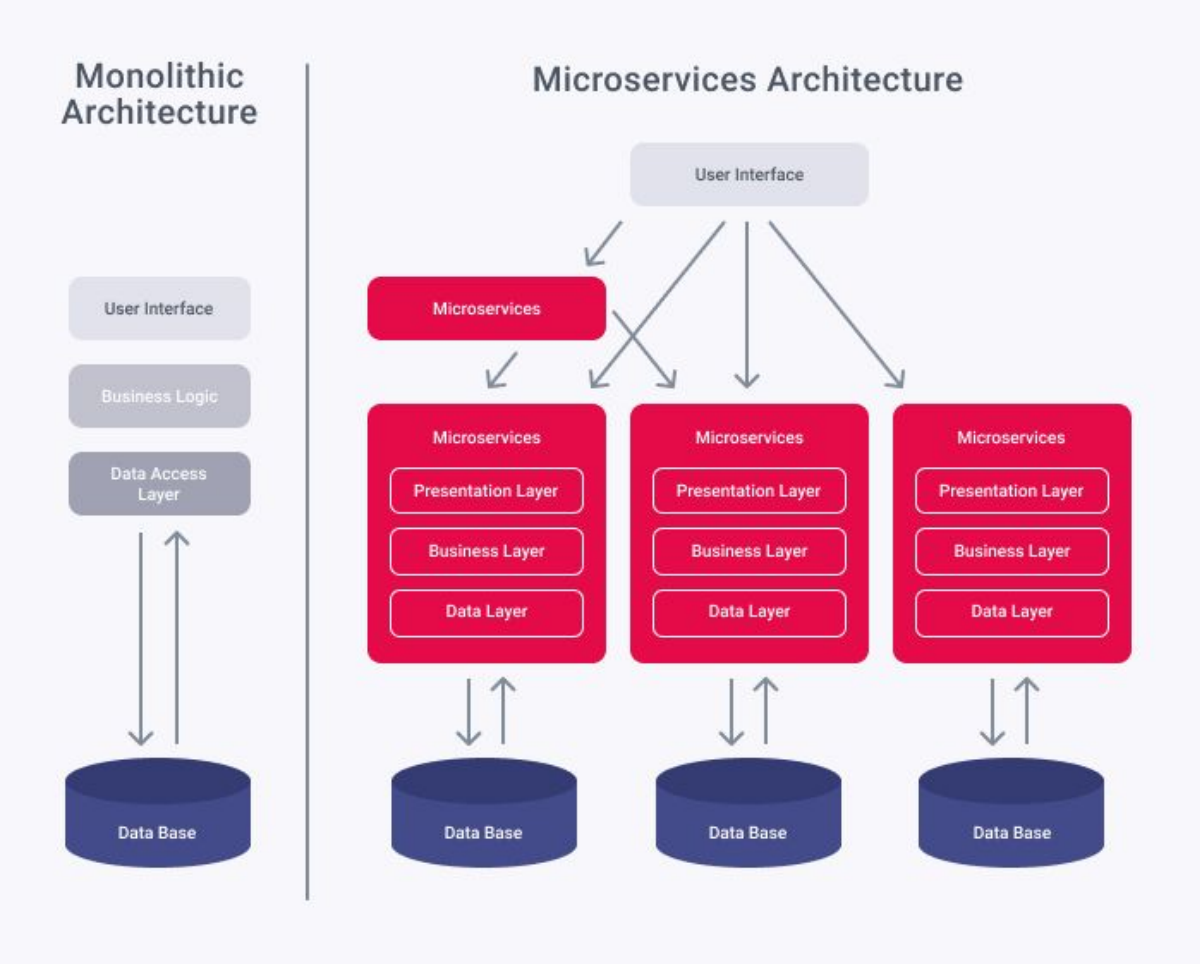
- Time To Market
- Evolutivité
- Fiabilité
- Difficulté à l'adoption de la technologie
- Manque de flexibilité
- Déploiement


What do we want?
Make it easier to deploy, maintain and update our services
How will we get it?
Microservices
Julie Lerman TheDataFarm.com @julielerman

Microservices – Kesako ?

Une architecture de microservices divise une app en une série de services déployables indépendamment qui communiquent via des **API**.

Chaque service individuel peut être déployé et mis à l'échelle de manière indépendante.


Monolithic Architecture
User Interface
Business Logic
Data Access Layer
Data Base
Microservices Architecture
User Interface
Microservices
Microservices
Presentation Layer
Business Layer
Data Layer
Microservices
Presentation Layer
Business Layer
Data Layer
Microservices
Presentation Layer
Business Layer
Data Layer
Data Base
Data Base
Data Base

Les avantages.

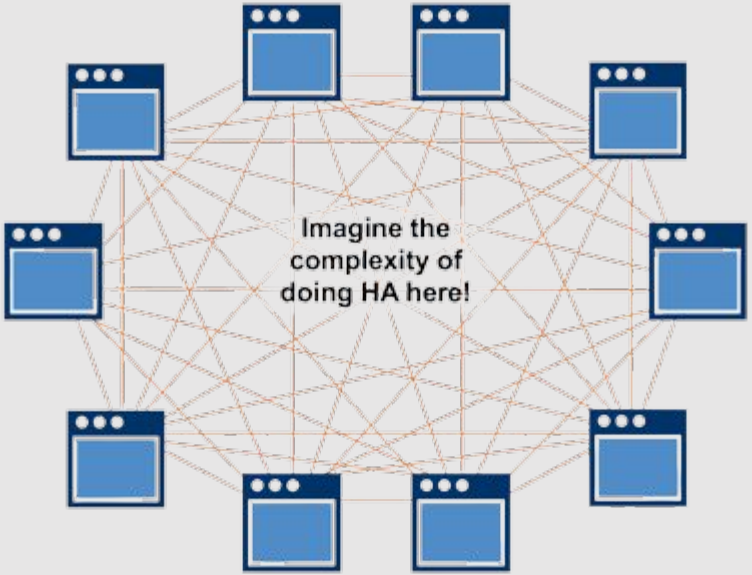
- Cycles de développement indépendants
- Builds et livraison simplifiés
- Configuration simplifiée
- Scalabilité horizontal
- Robustesse
- Équipe de travail minimale
- Evolutivité
- Fonctionnalité modulaire – modules indépendants
- Utilisation de conteneurs pour permettre un dev et un déploiement rapide

Les inconvénients.

- Identification et découpage des services et données
- Communication entre services
- Cohérence des données
- Event driven
- etc...

Mise en oeuvre de l’approche microservice

Communication inter-microservices

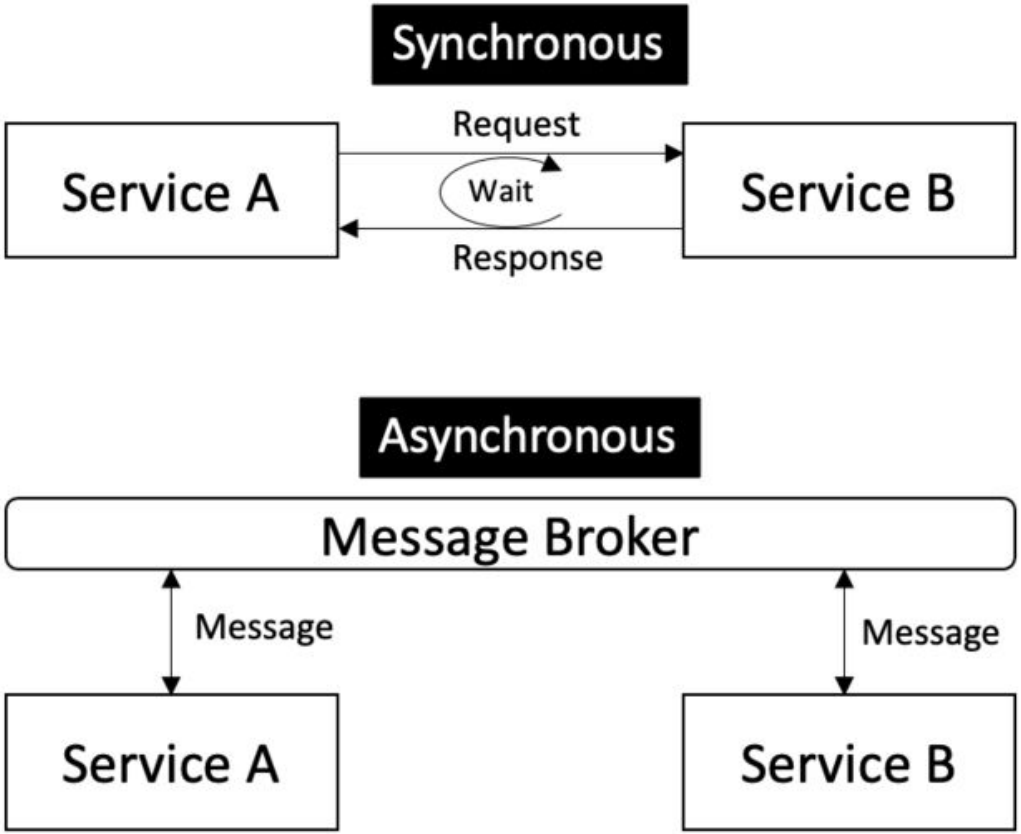


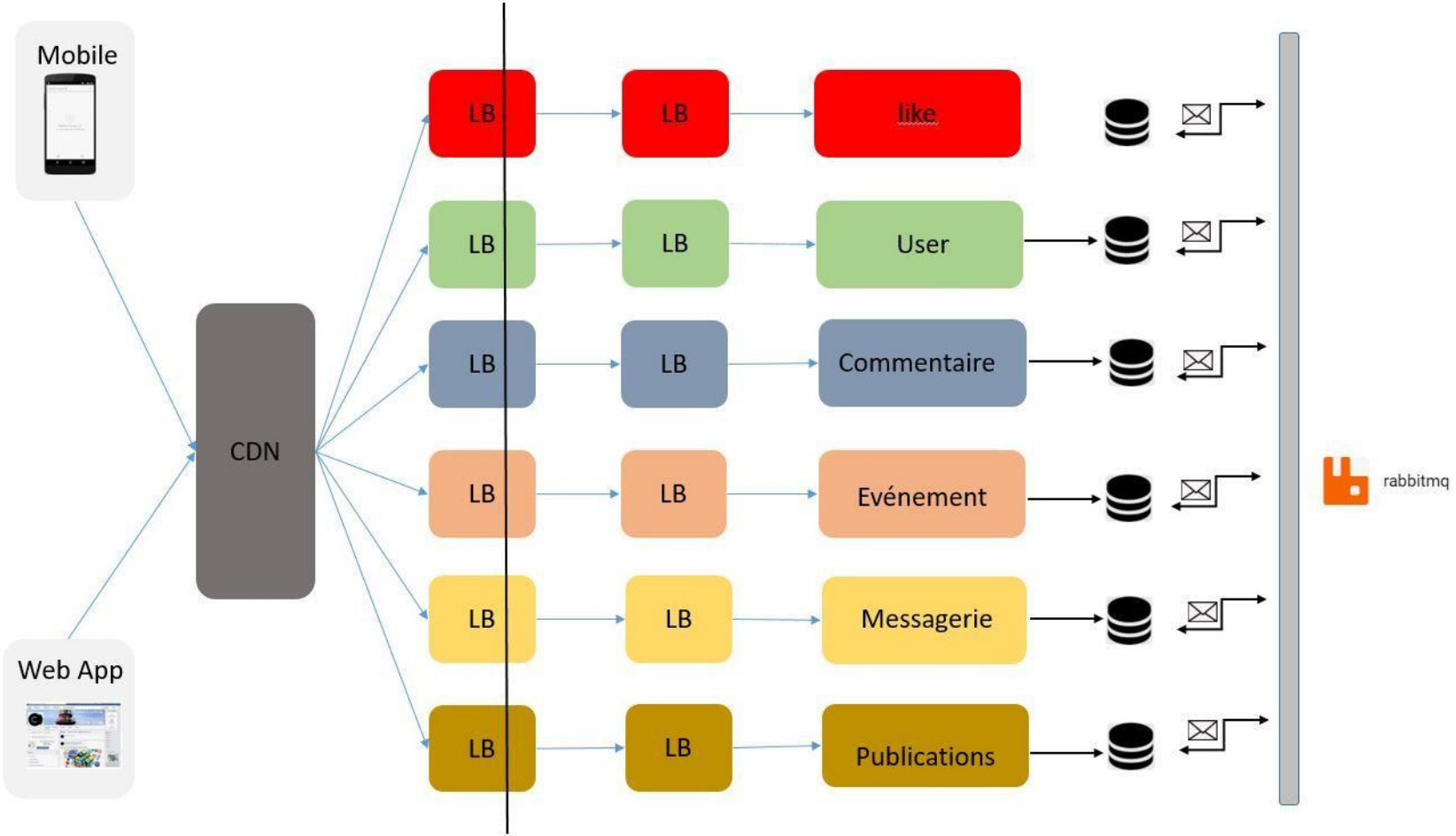
Communication inter-microservices.

La **communication inter-microservice** est l'un des grands défis d'une architecture orientée microservice. Il s'agit en clair du transfert de données entre microservices.

En effet deux possibilités s'offrent à nous :

- **Communication Synchrone HTTP**
- **Communication Asynchrone AMQP (cf. RabbitMQ sur l'architecture générale).**


Synchronous
Request
Service A
Wait
Service B
Response
Asynchronous
Message Broker
Message
Message
Service A
Service B


Mobile
Web App
CDN
LB
LB
like
LB
LB
User
LB
LB
Commentaire
LB
LB
Evénement
LB
LB
Messagerie
LB
LB
Publications
rabbitmq

Les solutions

Spring

Boot

Cloud

Netflix

Kubernetes & Istio

Spring Cloud

- Spring Cloud Stream
- Spring Cloud Azure
- Spring Cloud for Amazon Web Services
- Spring Cloud Bus
- Spring Cloud CLI
- Spring Cloud for Cloud Foundry
- Spring Cloud - Cloud Foundry Service Broker
- Spring Cloud Cluster
- Spring Cloud Commons
- Spring Cloud Config
- Spring Cloud Connectors
- Spring Cloud Consul
- Spring Cloud Contract
- Spring Cloud Function
- Spring Cloud Gateway
- Spring Cloud GCP
- Spring Cloud Netflix
- Spring Cloud Open Service Broker
- Spring Cloud Pipelines
- Spring Cloud Security
- Spring Cloud Sleuth
- Spring Cloud Stream App Starters
- Spring Cloud Task
- Spring Cloud Task App Starters
- Spring Cloud Vault
- Spring Cloud Zookeeper
- Spring Cloud App Broker
- Spring Cloud Circuit Breaker
- Spring Cloud Kubernetes
- Spring Cloud OpenFeign

Atelier : Mise en oeuvre d’une architecture microservices.

L'objectif de ce travail est de mettre en place plusieurs services indépendamment déployables qui communiquent entre eux, en utilisant les facilités offertes par **Spring Cloud** et **Spring Boot**.

Spring Cloud fournit des outils pour les développeurs pour construire rapidement et facilement des patrons communs de systèmes répartis.

Spring Boot permet de son côté de construire des applications Spring rapidement aussi rapidement que possible, en minimisant au maximum le temps de configuration.

Atelier : Mise en oeuvre d’une architecture microservices.

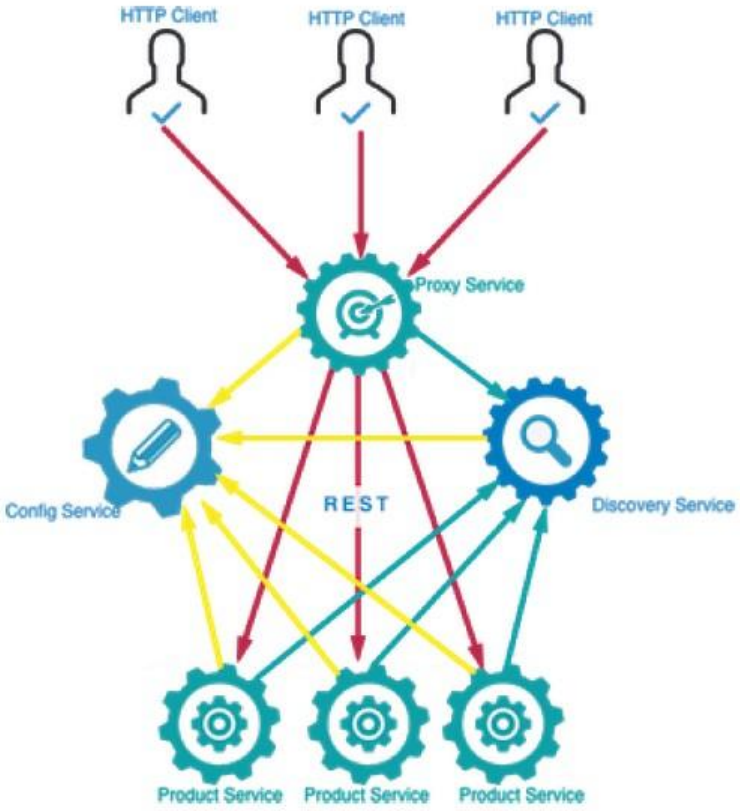
1. Product Service : Service principal, qui offre une API REST pour lister une liste de produits.

2. Configuration service : Service de configuration dont le rôle est de centraliser les fichiers de configuration des différents microservices.

3. Proxy Service : Passerelle se chargeant du routage d'une requête vers l'une des instances d'un service, de manière à gérer automatiquement la distribution de charge.

4. Discovery Service : Service permettant l'enregistrement des instances de services en vue d'être découvertes par d'autres services.

Architecture globale.



Caractéristiques du projet

Projet Maven avec **Java** et **Spring Boot**

Group: à définir

Artifact: à définir

Dépendances :

- Web
- Rest Repositories
- JPA : Java Persistence API
- Base de données pour le stockage
- Actuator : pour le monitoring et la gestion de l'application
- **Eureka Discovery** : pour l'intégration avec le **Discovery Service**
- Config client pour l'intégration avec config service.

Travaux-pratiques : Mise en oeuvre d'une architecture microservices.